EXPERIMENTS FROM 33 TO 40

33.

```
#include <stdio.h>

int main() {
    int pages[50], frames[10];
    int n, f, i, j, k;
    int faults = 0, hits = 0;
    int found, replace, farthest, nextUse;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    if (n <= 0 || n > 50) {
        printf("Invalid number of pages.\n");
        return 1;
    }

    printf("Enter page reference string:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    if (f <= 0 || f > 10) {
        printf("Invalid number of frames.\n");
```

```
    return 1;
}

for (i = 0; i < f; i++) {
    frames[i] = -1;
}

printf("\nPage\tFrames\t\tStatus\n");

for (i = 0; i < n; i++) {
    found = 0;

    for (j = 0; j < f; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    if (found) {
        hits++;
    } else {
        replace = -1;

        /* Find an empty frame first */
        for (j = 0; j < f; j++) {
            if (frames[j] == -1) {
                replace = j;
                break;
```

```
        }
    }

    /* Otherwise replace the page used farthest in future */
    if (replace == -1) {
        farthest = -1;

        for (j = 0; j < f; j++) {
            nextUse = n;

            for (k = i + 1; k < n; k++) {
                if (pages[k] == frames[j]) {
                    nextUse = k;
                    break;
                }
            }

            if (nextUse > farthest) {
                farthest = nextUse;
                replace = j;
            }
        }
    }

    frames[replace] = pages[i];
    faults++;
}

printf("%d\t", pages[i]);
```

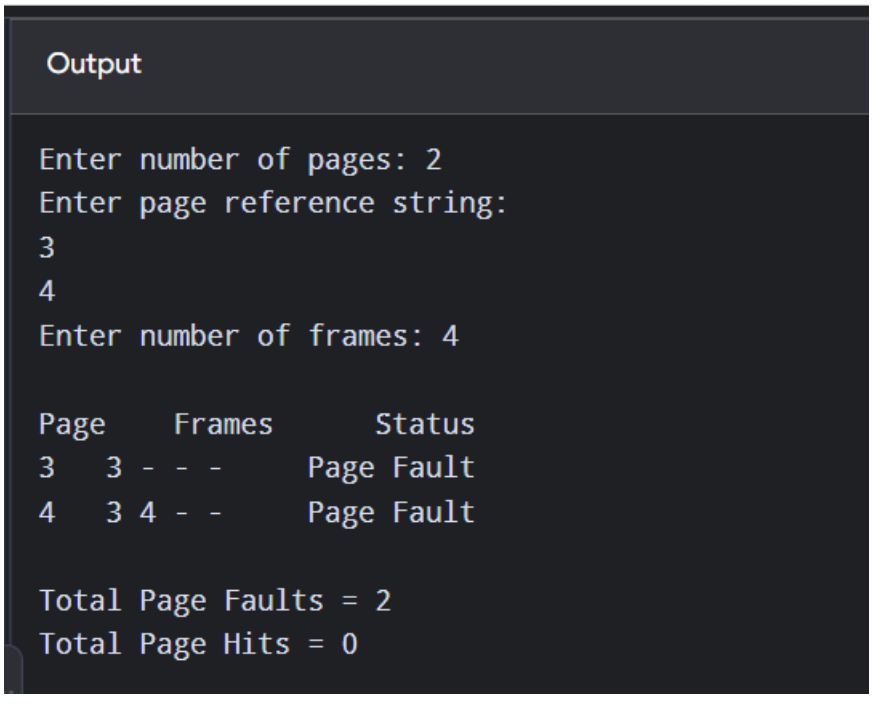
```
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        printf("\t%s\n", found ? "Hit" : "Page Fault");
    }

    printf("\nTotal Page Faults = %d\n", faults);
    printf("Total Page Hits = %d\n", hits);

    return 0;
}
```

OUTPUT:



34.

```
#include <stdio.h>

int main() {
    int disk[100] = {0};
    int start, length, i, j, choice;
    int allocated = 0;

    printf("CONTIGUOUS FILE ALLOCATION\n");

    while (1) {
        printf("\n1. Allocate File\n");
        printf("2. Display Disk Blocks\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter starting block (0-99): ");
                scanf("%d", &start);

                printf("Enter file length: ");
                scanf("%d", &length);

                if (start < 0 || start >= 100 ||
                    length <= 0 || length > 100 - start) {
                    printf("Invalid block range or file length.\n");
```

```
            break;
        }

        allocated = 1;

        for (i = start; i < start + length; i++) {
            if (disk[i] != 0) {
                allocated = 0;
                break;
            }
        }

        if (allocated) {
            for (i = start; i < start + length; i++) {
                disk[i] = 1;
            }
            printf("File allocated successfully.\n");
            printf("Allocated blocks: ");
            for (i = start; i < start + length; i++) {
                printf("%d ", i);
            }
            printf("\n");
        } else {
            printf("Allocation failed: blocks are occupied.\n");
        }
        break;

    case 2:
        printf("\nBlock\tStatus\n");
```

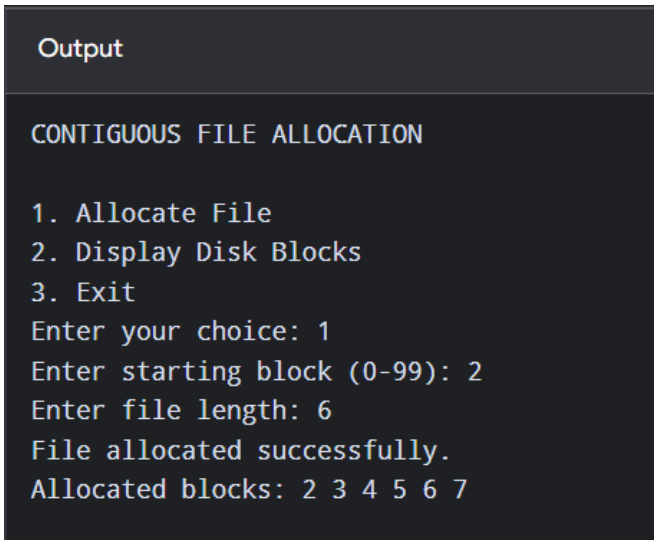
```
            for (i = 0; i < 100; i++) {
                if (disk[i] == 1)
                    printf("%d\tAllocated\n", i);
                else
                    printf("%d\tFree\n", i);
            }
            break;

        case 3:
            printf("Exiting program.\n");
            return 0;

        default:
            printf("Invalid choice.\n");
    }
  }

  return 0;
}
```

OUTPUT:



35.

```
#include <stdio.h>

#define MAX_BLOCKS 100
#define MAX_FILES 20

int main() {
    int disk[MAX_BLOCKS] = {0};
    int indexBlock[MAX_FILES];
    int dataBlocks[MAX_FILES][MAX_BLOCKS];
    int fileCount = 0;
    int choice, i, j, index, n, block, valid;

    while (1) {
        printf("\n--- Indexed File Allocation ---\n");
        printf("1. Allocate File\n");
        printf("2. Display Files\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (fileCount >= MAX_FILES) {
                    printf("Maximum number of files reached.\n");
                    break;
                }
```

```
printf("Enter index block (0-99): ");
scanf("%d", &index);

if (index < 0 || index >= MAX_BLOCKS ||
    disk[index] == 1) {
    printf("Invalid or occupied index block.\n");
    break;
}

printf("Enter number of data blocks: ");
scanf("%d", &n);

if (n <= 0 || n >= MAX_BLOCKS) {
    printf("Invalid number of blocks.\n");
    break;
}

valid = 1;

for (i = 0; i < n; i++) {
    printf("Enter data block %d: ", i + 1);
    scanf("%d", &block);

    if (block < 0 || block >= MAX_BLOCKS ||
        disk[block] == 1 || block == index) {
        printf("Invalid or occupied block. Try again.\n");
        i--;
        continue;
    }
```

```
    for (j = 0; j < i; j++) {
        if (dataBlocks[fileCount][j] == block) {
            break;
        }
    }

    if (j < i) {
        printf("Duplicate block. Enter another block.\n");
        i--;
        continue;
    }

    dataBlocks[fileCount][i] = block;
}

disk[index] = 1;

for (i = 0; i < n; i++) {
    disk[dataBlocks[fileCount][i]] = 1;
}

indexBlock[fileCount] = index;
fileCount++;

printf("File allocated successfully.\n");
printf("Index block: %d\n", index);
printf("Data blocks: ");
```

```
    for (i = 0; i < n; i++) {
        printf("%d ", dataBlocks[fileCount - 1][i]);
    }

    printf("\n");
    break;

case 2:
    if (fileCount == 0) {
        printf("No files allocated.\n");
        break;
    }

    printf("\nFile\tIndex Block\tData Blocks\n");

    for (i = 0; i < fileCount; i++) {
        printf("%d\t%d\t\t", i + 1, indexBlock[i]);

        for (j = 0; j < MAX_BLOCKS; j++) {
            if (disk[j] == 1) {
                int isData = 0;
                int k;

                for (k = 0; k < MAX_BLOCKS; k++) {
                    if (k == indexBlock[i]) {
                        continue;
                    }
                }
```

```
            for (k = 0; k < MAX_BLOCKS; k++) {
                if (k == indexBlock[i]) {
                    isData = 0;
                    break;
                }
            }
        }
    }

    /* Print this file's data blocks */
    for (j = 0; j < MAX_BLOCKS; j++) {
        if (j == indexBlock[i]) {
            continue;
        }

        int k;
        for (k = 0; k < MAX_BLOCKS; k++) {
            if (k < MAX_BLOCKS &&
                dataBlocks[i][k] == j) {
                printf("%d ", j);
                break;
            }
        }
    }

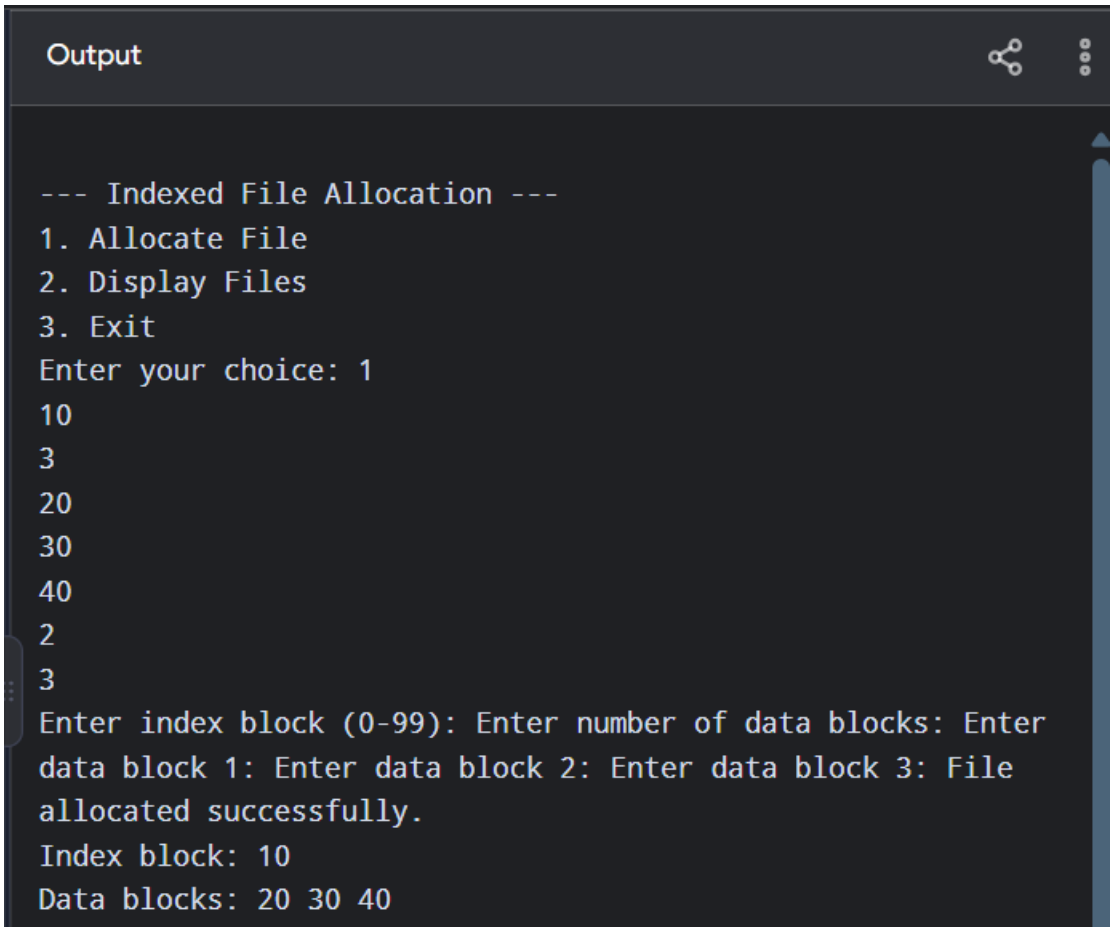
    printf("\n");
}
break;
```

```
            case 3:
                printf("Exiting program.\n");
                return 0;

            default:
                printf("Invalid choice.\n");
        }
    }

    return 0;
}
```

OUTPUT:



36.

```
#include <stdio.h>

#define MAX 100

int main() {
    int disk[MAX] = {0};
    int fileBlocks[MAX][MAX];
    int blockCount[MAX];
    int fileCount = 0;
    int choice, i, j, block, n, valid;

    while (1) {
        printf("\n--- Linked File Allocation ---\n");
        printf("1. Allocate File\n");
        printf("2. Display Files\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (fileCount >= MAX) {
                    printf("Maximum files reached.\n");
                    break;
                }

                printf("Enter number of blocks required: ");
```

```
scanf("%d", &n);

if (n <= 0 || n > MAX) {
    printf("Invalid number of blocks.\n");
    break;
}

valid = 1;

for (i = 0; i < n; i++) {
    printf("Enter block number %d (0-99): ", i + 1);
    scanf("%d", &block);

    if (block < 0 || block >= MAX || disk[block] == 1) {
        printf("Invalid or occupied block. Try again.\n");
        i--;
        continue;
    }

    for (j = 0; j < i; j++) {
        if (fileBlocks[fileCount][j] == block)
            break;
    }

    if (j < i) {
        printf("Duplicate block. Try another.\n");
        i--;
        continue;
    }
```

```
            fileBlocks[fileCount][i] = block;
        }

        for (i = 0; i < n; i++)
            disk[fileBlocks[fileCount][i]] = 1;

        blockCount[fileCount] = n;
        fileCount++;

        printf("File allocated successfully.\n");
        break;

    case 2:
        if (fileCount == 0) {
            printf("No files allocated.\n");
            break;
        }

        printf("\nFile\tBlocks\t\tLinked Order\n");

        for (i = 0; i < fileCount; i++) {
            printf("%d\t%d\t\t", i + 1, blockCount[i]);

            for (j = 0; j < blockCount[i]; j++) {
                printf("%d", fileBlocks[i][j]);

                if (j < blockCount[i] - 1)
                    printf(" -> ");
```

```
                }

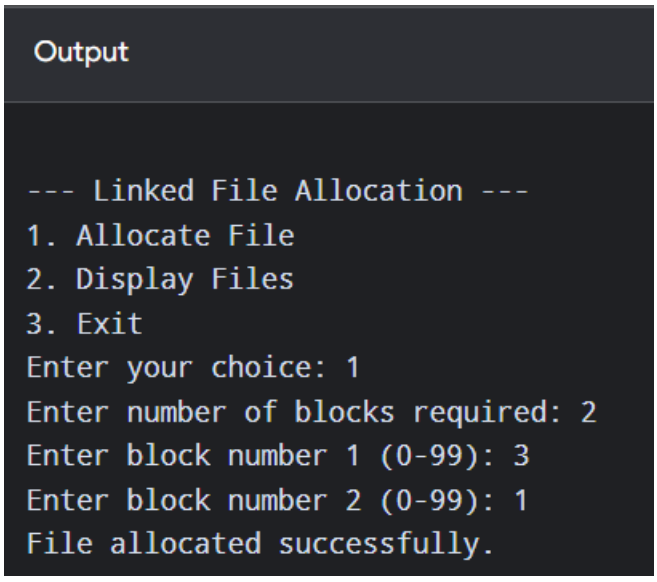
                printf(" -> NULL\n");
            }
            break;

        case 3:
            printf("Exiting program.\n");
            return 0;

        default:
            printf("Invalid choice.\n");
    }
  }

  return 0;
}
```

OUTPUT:



37.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i, head, total = 0;
    int request[100];

    printf("Enter number of disk requests: ");
    scanf("%d", &n);

    if (n <= 0 || n > 100) {
        printf("Invalid number of requests.\n");
        return 0;
    }

    printf("Enter disk request sequence: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &request[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("\nDisk head movement:\n");
    printf("%d", head);

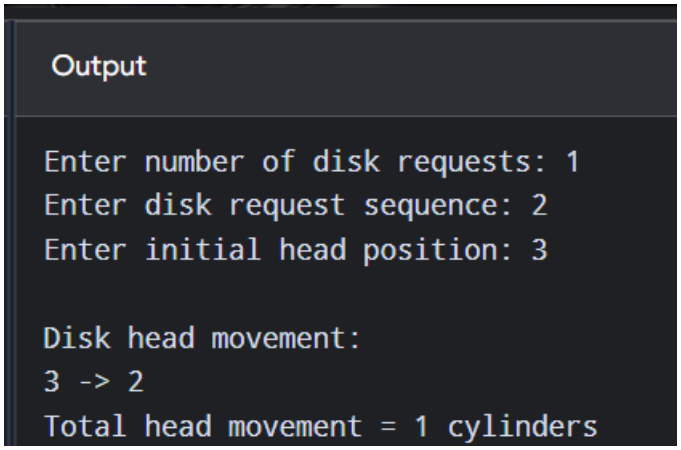
    for (i = 0; i < n; i++) {
```

```
        total += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }

    printf("\nTotal head movement = %d cylinders\n", total);

    return 0;
}
```

OUTPUT:



38.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[100], n, head, diskSize, direction;
    int i, j, temp, total = 0;

    printf("Enter number of disk requests: ");
    scanf("%d", &n);

    if (n <= 0 || n > 100) {
        printf("Invalid number of requests.\n");
        return 0;
    }

    printf("Enter disk request sequence: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &request[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size (number of cylinders): ");
    scanf("%d", &diskSize);

    if (diskSize <= 0 || head < 0 || head >= diskSize) {
```

```
    printf("Invalid disk size or head position.\n");
    return 0;
}

for (i = 0; i < n; i++) {
    if (request[i] < 0 || request[i] >= diskSize) {
        printf("Invalid disk request.\n");
        return 0;
    }
}

printf("Enter direction (0 = left, 1 = right): ");
scanf("%d", &direction);

if (direction != 0 && direction != 1) {
    printf("Invalid direction.\n");
    return 0;
}

/* Sort requests */
for (i = 0; i < n - 1; i++) {
    for (j = 0; j < n - i - 1; j++) {
        if (request[j] > request[j + 1]) {
            temp = request[j];
            request[j] = request[j + 1];
            request[j + 1] = temp;
        }
    }
}
```

```
printf("\nDisk head movement:\n");
printf("%d", head);

if (direction == 0) {
    /* Move left */
    for (i = n - 1; i >= 0; i--) {
        if (request[i] < head) {
            total += abs(head - request[i]);
            head = request[i];
            printf(" -> %d", head);
        }
    }

    /* Reach the left end */
    total += head;
    head = 0;
    printf(" -> %d", head);

    /* Move right */
    for (i = 0; i < n; i++) {
        if (request[i] > head) {
            total += abs(head - request[i]);
            head = request[i];
            printf(" -> %d", head);
        }
    }
} else {
    /* Move right */
```

```
        for (i = 0; i < n; i++) {
            if (request[i] > head) {
                total += abs(head - request[i]);
                head = request[i];
                printf(" -> %d", head);
            }
        }

        /* Reach the right end */
        total += (diskSize - 1) - head;
        head = diskSize - 1;
        printf(" -> %d", head);

        /* Move left */
        for (i = n - 1; i >= 0; i--) {
            if (request[i] < head) {
                total += abs(head - request[i]);
                head = request[i];
                printf(" -> %d", head);
            }
        }
    }

    printf("\nTotal head movement = %d cylinders\n", total);

    return 0;
}
```

OUTPUT:

```
Output

Enter number of disk requests: 1
Enter disk request sequence: 2
Enter initial head position: 3
Enter disk size (number of cylinders): 4
Enter direction (0 = left, 1 = right): 1

Disk head movement:
3 -> 3 -> 2
Total head movement = 1 cylinders
```

39.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[100], n, head, diskSize, i, j, temp;
    int total = 0, start;

    printf("Enter number of disk requests: ");
    scanf("%d", &n);

    if (n <= 0 || n > 100) {
        printf("Invalid number of requests.\n");
        return 0;
    }

    printf("Enter disk request sequence: ");
```

```
for (i = 0; i < n; i++) {
    scanf("%d", &request[i]);
}

printf("Enter initial head position: ");
scanf("%d", &head);

printf("Enter disk size (number of cylinders): ");
scanf("%d", &diskSize);

if (diskSize <= 0 || head < 0 || head >= diskSize) {
    printf("Invalid disk size or head position.\n");
    return 0;
}

for (i = 0; i < n; i++) {
    if (request[i] < 0 || request[i] >= diskSize) {
        printf("Invalid disk request.\n");
        return 0;
    }
}

/* Sort requests in ascending order */
for (i = 0; i < n - 1; i++) {
    for (j = 0; j < n - i - 1; j++) {
        if (request[j] > request[j + 1]) {
            temp = request[j];
            request[j] = request[j + 1];
            request[j + 1] = temp;
```

```
        }
    }
}

printf("\nDisk head movement:\n");
printf("%d", head);

start = head;

/* Service requests to the right */
for (i = 0; i < n; i++) {
    if (request[i] >= start) {
        total += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
}

/* Move to the last cylinder */
if (head != diskSize - 1) {
    total += (diskSize - 1) - head;
    head = diskSize - 1;
    printf(" -> %d", head);
}

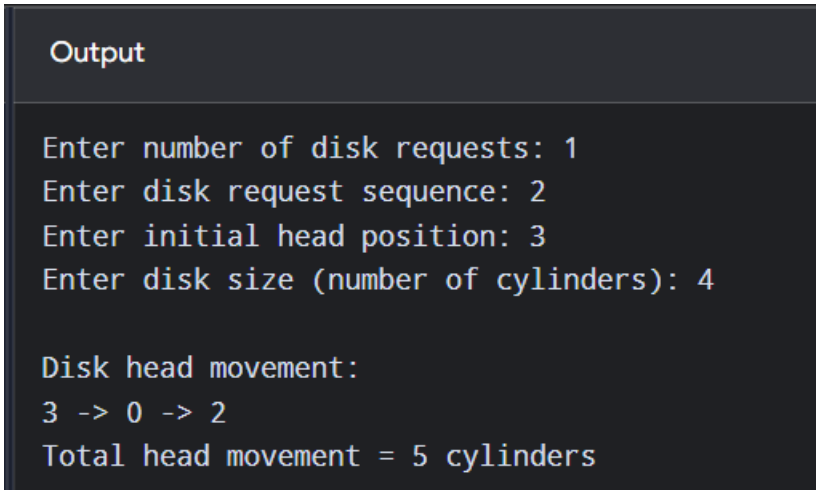
/* Circular jump to cylinder 0 */
total += diskSize - 1;
head = 0;
printf(" -> %d", head);
```

```
    /* Service remaining requests from the beginning */
    for (i = 0; i < n; i++) {
        if (request[i] < start) {
            total += abs(head - request[i]);
            head = request[i];
            printf(" -> %d", head);
        }
    }

    printf("\nTotal head movement = %d cylinders\n", total);

    return 0;
}
```

OUTPUT:



40.

```
#include <stdio.h>

int main() {
    int owner, group, others;

    printf("Linux File Permissions\n");
    printf("----------------------\n");

    printf("Enter owner permissions (0-7): ");
    scanf("%d", &owner);

    printf("Enter group permissions (0-7): ");
    scanf("%d", &group);

    printf("Enter others permissions (0-7): ");
    scanf("%d", &others);

    if (owner < 0 || owner > 7 ||
        group < 0 || group > 7 ||
        others < 0 || others > 7) {
        printf("Invalid permissions! Enter values from 0 to 7.\n");
        return 0;
    }

    printf("\nPermission Details\n");
    printf("------------------\n");
    printf("Owner : %d : ", owner);
```

```
    printf("%c", (owner & 4) ? 'r' : '-');
    printf("%c", (owner & 2) ? 'w' : '-');
    printf("%c\n", (owner & 1) ? 'x' : '-');

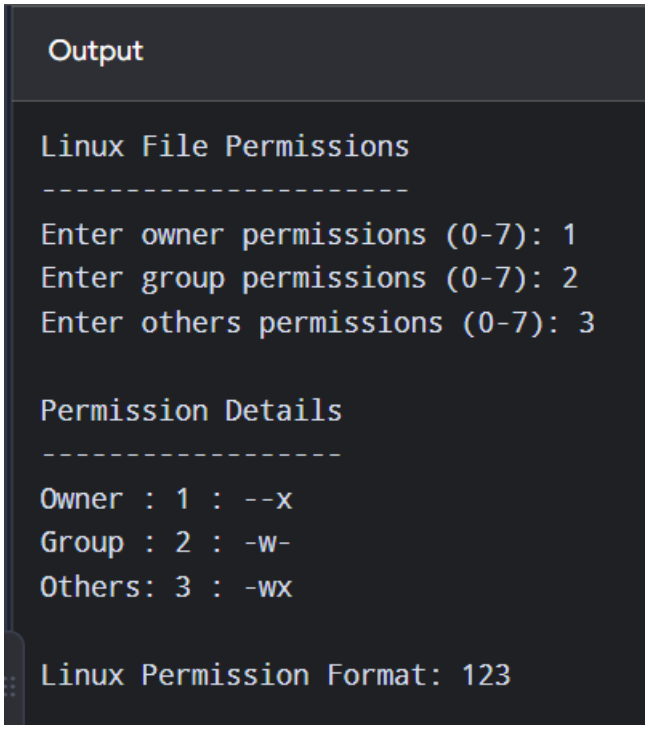
    printf("Group : %d : ", group);
    printf("%c", (group & 4) ? 'r' : '-');
    printf("%c", (group & 2) ? 'w' : '-');
    printf("%c\n", (group & 1) ? 'x' : '-');

    printf("Others: %d : ", others);
    printf("%c", (others & 4) ? 'r' : '-');
    printf("%c", (others & 2) ? 'w' : '-');
    printf("%c\n", (others & 1) ? 'x' : '-');

    printf("\nLinux Permission Format: ");
    printf("%d%d%d\n", owner, group, others);

    return 0;
}
```

OUTPUT:


Output
Linux File Permissions
Enter owner permissions (0-7): 1
Enter group permissions (0-7): 2
Enter others permissions (0-7): 3
Permission Details
Owner : 1 : --x
Group : 2 : -w-
Others: 3 : -wx
Linux Permission Format: 123